ESILV

Class: **ACTIF 4**

Professor: **Rougui Imad**

Project Report

4. Simple Swap Curve, Cash, Futures, Swap Bootstrapping

(VBA + Excel)

Matthieu HANNA GERGUIS

Submission date: **31 December 2025**

Contents

1 Project overview **2**

2 Financial logic and formulas **2**

3 Inputs sheet **3**

4 Curve sheet **3**

5 Conclusion **4**

A VBA code **4**

A.1 Inputs macro (CalcInputs_VBA_Outputs) 4

A.2 Curve macro (CalcCurve_VBA_Outputs) 7

1 Project overview

The goal of this project is to build a simple interest rate curve from a small set of market quotes. I used cash deposits for short maturities, futures for the middle part of the curve, and swaps for longer maturities. The final outputs of the curve are discount factors and continuously compounded zero rates.

My main strategy was to make the work very clear and easy to check. I did not want to only show final numbers. I wanted to show the full reasoning, step by step, and also prove that the same reasoning works in VBA. This is why each sheet is built in two layers. The first layer is the main Excel table with formulas. The second layer is a "VBA Results" table placed below, where a macro writes the same derived values as plain numbers. At the end, a simple check shows that Excel and VBA match.

2 Financial logic and formulas

I start from a valuation date t_0. For each instrument i, I define a start date t_{i-1}, an end date t_i, an accrual factor τ_i and a maturity T_i in years. The accrual factor depends on the day count convention. I computed it in two ways. First, I use `YEARFRAC` as a reference. Second, I compute a manual accrual to show the real financial formula:

$$\tau_i = \begin{cases} \dfrac{t_i - t_{i-1}}{360} & \text{ACT/360} \\ \dfrac{\text{DAYS360}(t_{i-1}, t_i)}{360} & 30/360 \\ \dfrac{t_i - t_{i-1}}{365} & \text{ACT/365} \end{cases}$$

For swaps with an annual fixed leg, I use $\alpha_i = 1$. For cash and futures, I use $\alpha_i = \tau_i$ because the payoff is proportional to the accrual.

The quote is not the same product for all rows, so I convert it into a consistent rate. For cash deposits, the quote is a rate in percent, so $r = \text{Quote}/100$. For futures, the quote is a price P, so the implied forward rate is $f = (100 - P)/100$. For swaps, the quote is a fixed rate in percent, so $S = \text{Quote}/100$.

To build the curve, I bootstrap discount factors. For cash deposits, I use the money-market formula:

$$DF = \frac{1}{1 + r\tau}$$

For futures, I chain discount factors from one period to the next:

$$DF_i = \frac{DF_{i-1}}{1 + f_i \tau_i}$$

For swaps (annual fixed leg), I use the par swap relationship. If N is the last payment year, the last discount factor is:

$$DF_N = \frac{1 - S \sum_{k=1}^{N-1} (\alpha_k DF_k)}{1 + S\alpha_N}$$

Finally, once $DF(T)$ is known, I compute the continuously compounded zero rate:

$$z(T) = -\frac{\ln(DF(T))}{T}$$

3 Inputs sheet

On the Inputs sheet, I organized the information so that the first columns are pure inputs (Type, Tenor, Quote, DayCount). Then I placed all derived columns next to them: the basis code, the tenor in months, the start date, the end date, the accrual factors, the coupon factor, and the maturity in years. This is the first important part of the project because all later calculations depend on dates and accruals.

A key point is how I treated futures. Instead of using the valuation date as start date, I used the previous instrument end date as the next future start date. Then I set the future end date as start date plus three months. This creates a clean sequence of quarterly periods, which is consistent with the usual way futures are linked when we bootstrap short-term curves.

To show that the work is not a black box, I kept both accrual computations: `YEARFRAC` and the manual day count formulas. This makes it easy to explain and also easy to debug. After that, I created a VBA Results table below the main table. The macro `CalcInputs_VBA_Outputs` recomputes the same derived columns and writes the values. The last column checks that the VBA values match the Excel values.

4. Simple Swap Curve, Cash, Futures, Swap Bootstrapping (VBA + Excel)

Valuation Date	31/12/2025
Notes	The quotes used in this case study are illustrative (invented) values, constructed to be internally consistent and realistic for the exercise.

'Type c'(CASH; FUT; SWAP)	*'Tenor c'(1M;3M;...;10Y)*	*'CASH: r (%) ; FUT: P ; SWAP: S (%)*	*'Day count: ACT/360; 30/360; ACT/365*	*'BasisCode c' (0;2;3)*	*'Fixed leg freq: m (annual=1)*	*'TenorMonths + if(Y) 12 = ELSE n*	*'Start: t0 (or t_{i-1} for FUT)*	*'End: t0+Tenor (FUT: +3M)*	*'τ_i = DayCountFrac(t_{i-1},t_i)*	*'τ_{manual} = {(t_i−t_{i-1})/360 ; DAYS360/360 ; (t_i−t_{i-1})/365 }*	*'α_i = {1 if SWAP & annual ; else τ_i}*	*T = YearFrac(t0,t_i) (or TenorMonths/12)*
Type	Tenor	Quote	DayCount	BasisCode	FixedFreq	TenorMonths	StartDate	EndDate	Accrual τ (YearFrac)	Accrual τ (manual)	Coupon α	T (years)
CASH	1M	5,1	ACT/360	2		1	31/12/2025	31/01/2026	0,086111111	0,086111111	0,086111111	0,083333333
CASH	3M	5,2	ACT/360	2		3	31/12/2025	31/03/2026	0,25	0,25	0,25	0,25
CASH	6M	5,25	ACT/360	2		6	31/12/2025	30/06/2026	0,502777778	0,502777778	0,502777778	0,5
FUT	9M	94,8	ACT/360	2		9	30/06/2026	30/09/2026	0,255555556	0,255555556	0,255555556	0,75
FUT	12M	94,6	ACT/360	2		12	30/09/2026	30/12/2026	0,252777778	0,252777778	0,252777778	1
FUT	15M	94,4	ACT/360	2		15	30/12/2026	30/03/2027	0,25	0,25	0,25	1,25
FUT	18M	94,2	ACT/360	2		18	30/03/2027	30/06/2027	0,255555556	0,255555556	0,255555556	1,5
FUT	21M	94	ACT/360	2		21	30/06/2027	30/09/2027	0,255555556	0,255555556	0,255555556	1,75
FUT	24M	93,8	ACT/360	2		24	30/09/2027	30/12/2027	0,252777778	0,252777778	0,252777778	2
SWAP	3Y	4,2	30/360	0	1	36	31/12/2025	31/12/2028	3	3	1	3
SWAP	4Y	4,1	30/360	0	1	48	31/12/2025	31/12/2029	4	4	1	4
SWAP	5Y	4,05	30/360	0	1	60	31/12/2025	31/12/2030	5	5	1	5
SWAP	7Y	4	30/360	0	1	84	31/12/2025	31/12/2032	7	7	1	7
SWAP	10Y	3,95	30/360	0	1	120	31/12/2025	31/12/2035	10	10	1	10

DayCount	BasisCode
ACT/360	2
30/360	0
ACT/365	3

VBA Results - Inputs Preprocessing (values written by macro)

Run the macro CalcInputs_VBA_Outputs to compute derived columns with VBA and compare against Excel formulas.

Type	Tenor	Quote	DayCount	BasisCode (VBA)	TenorMonths (VBA)	StartDate (VBA)	EndDate (VBA)	Accrual τ (YearFrac VBA)	Accrual τ (manual VBA)	Coupon α (VBA)	T (years) (VBA)	Check (VBA vs Excel)
CASH	1M	5,1	ACT/360	2	1	31/12/2025	31/01/2026	0,086111111	0,086111111	0,086111111	0,083333333	OK
CASH	3M	5,2	ACT/360	2	3	31/12/2025	31/03/2026	0,250000000	0,250000000	0,250000000	0,250000000	OK
CASH	6M	5,25	ACT/360	2	6	31/12/2025	30/06/2026	0,502777778	0,502777778	0,502777778	0,500000000	OK
FUT	9M	94,8	ACT/360	2	9	30/06/2026	30/09/2026	0,255555556	0,255555556	0,255555556	0,750000000	OK
FUT	12M	94,6	ACT/360	2	12	30/09/2026	30/12/2026	0,252777778	0,252777778	0,252777778	1,000000000	OK
FUT	15M	94,4	ACT/360	2	15	30/12/2026	30/03/2027	0,250000000	0,250000000	0,250000000	1,250000000	OK
FUT	18M	94,2	ACT/360	2	18	30/03/2027	30/06/2027	0,255555556	0,255555556	0,255555556	1,500000000	OK
FUT	21M	94	ACT/360	2	21	30/06/2027	30/09/2027	0,255555556	0,255555556	0,255555556	1,750000000	OK
FUT	24M	93,8	ACT/360	2	24	30/09/2027	30/12/2027	0,252777778	0,252777778	0,252777778	2,000000000	OK
SWAP	3Y	4,2	30/360	0	36	31/12/2025	31/12/2028	3,000000000	3,000000000	1,000000000	3,000000000	OK
SWAP	4Y	4,1	30/360	0	48	31/12/2025	31/12/2029	4,000000000	4,000000000	1,000000000	4,000000000	OK
SWAP	5Y	4,05	30/360	0	60	31/12/2025	31/12/2030	5,000000000	5,000000000	1,000000000	5,000000000	OK
SWAP	7Y	4	30/360	0	84	31/12/2025	31/12/2032	7,000000000	7,000000000	1,000000000	7,000000000	OK
SWAP	10Y	3,95	30/360	0	120	31/12/2025	31/12/2035	10,000000000	10,000000000	1,000000000	10,000000000	OK

Figure 1: Inputs sheet with the Excel table on top and the VBA Results validation table below.

4 Curve sheet

On the Curve sheet, I use the Inputs information to build the curve outputs. I first compute a single column called RateUsed to make the next formulas consistent. Then I bootstrap discount factors in the correct order. I start with cash, then I continue with futures by chaining discount factors period by period, and finally I use swaps to extend the curve to long maturities.

The swap part is the most important step because it requires the sum of discounted fixed coupons. Because the fixed leg is annual, I need discount factors at integer years like 1Y, 2Y, 3Y, and so on. In Excel, I created a helper table "DF at Year". It returns the discount factor for each year using the last known maturity T that is smaller or equal to the integer year. This is a simple and readable approach for a class project, and I reproduced the same idea in VBA with the function `DFAtYear`.

Just like on the Inputs sheet, I built a VBA Results table below the main curve table. The macro `CalcCurve_VBA_Outputs` writes RateUsed, discount factors, and zero rates as values. I then compare the VBA results with the Excel results. To make the validation very visible, I

added a small box on the right with the status PASS and the maximum absolute difference in discount factors. This gives a quick proof that the implementations are consistent.

4. Simple Swap Curve, Cash, Futures, Swap Bootstrapping (VBA + Excel)

Valuation Date	31/12/2025
Notes	The quotes used in this case study are illustrative (invented) values, constructed to be internally consistent and realistic for the exercise.

Validation	PASS
Max |ΔDF|	1,11022E-16

'Type ∈ {CASH;FUT;SWAP}	'Tenor ∈ {1M;...;10Y}	'Quote: CASH r(%); FUT P; SWAP S(%)	'DayCount ∈ {ACT/360;30/360;ACT/365}	'Start: t_{i-1} (FUT: prev End)	'End: t_i	'τ_i(manual): { t_i-t_{i-1}}/360 ; DAYS360/360 ; (t_i-t_{i-1})/365}	'α_i = (1 if SWAP & annual; else τ_i)	'T_i = YearFrac(t0,t_i) (or TenorMonths/12)	'RateUsed: r=Quote/100; P=(100−P)/100, S=Quote/100	'CASH: DF=1/(1+r·τ) | FUT: DF_i=DF_{i−1}/(1+r·τ) | SWAP: DF_N=(1−S·Σ_{k=1..N−1}(α_k DF_k))/(1+S·α_N)	'Zero(cc): z(T)=−ln(DF)/T	Formula reference (text)

Type	Tenor	Quote	DayCount	StartDate	EndDate	Accrual τ (manual)	Coupon α	T (years)	Rate Used	Discount Factor DF (Excel)	Zero Rate (cc)	Math formula
CASH	1M	5,1	ACT/360	31/12/2025	31/01/2026	0,086111111	0,086111111	0,083333333	0,051	0,995627536	0,052584617	DF = 1/(1+r·τ)
CASH	3M	5,2	ACT/360	31/12/2025	31/03/2026	0,25	0,25	0,25	0,052	0,987166831	0,051664901	DF = 1/(1+r·τ)
CASH	6M	5,25	ACT/360	31/12/2025	30/06/2026	0,502777778	0,502777778	0,5	0,0525	0,974282989	0,05210695	DF = 1/(1+r·τ)
FUT	9M	94,8	ACT/360	30/06/2026	30/09/2026	0,255555556	0,255555556	0,75	0,052	0,961505647	0,052339788	DF_i = DF_{i-1}/(1+f_{i-1}·τ_i)
FUT	12M	94,6	ACT/360	30/09/2026	30/12/2026	0,252777778	0,252777778	1	0,054	0,948557632	0,052812519	DF_i = DF_{i-1}/(1+f_{i-1}·τ_i)
FUT	15M	94,4	ACT/360	30/12/2026	30/03/2027	0,25	0,25	1,25	0,056	0,935461373	0,053372339	DF_i = DF_{i-1}/(1+f_{i-1}·τ_i)
FUT	18M	94,2	ACT/360	30/03/2027	30/06/2027	0,255555556	0,255555556	1,5	0,058	0,921796274	0,054286914	DF_i = DF_{i-1}/(1+f_{i-1}·τ_i)
FUT	21M	94	ACT/360	30/06/2027	30/09/2027	0,255555556	0,255555556	1,75	0,06	0,907877486	0,055226159	DF_i = DF_{i-1}/(1+f_{i-1}·τ_i)
FUT	24M	93,8	ACT/360	30/09/2027	30/12/2027	0,252777778	0,252777778	2	0,062	0,893868579	0,056098259	DF_i = DF_{i-1}/(1+f_{i-1}·τ_i)
SWAP	3Y	4,2	30/360	31/12/2025	31/12/2028	3	1	3	0,042	0,885430029	0,040560614	DF_N = (1−S·Σ_{k=1..N−1}(α_k·DF_k))/(1+S·α_N)
SWAP	4Y	4,1	30/360	31/12/2025	31/12/2029	4	1	4	0,041	0,853177604	0,039696885	DF_N = (1−S·Σ_{k=1..N−1}(α_k·DF_k))/(1+S·α_N)
SWAP	5Y	4,05	30/360	31/12/2025	31/12/2030	5	1	5	0,0405	0,821689689	0,039278492	DF_N = (1−S·Σ_{k=1..N−1}(α_k·DF_k))/(1+S·α_N)
SWAP	7Y	4	30/360	31/12/2025	31/12/2032	7	1	7	0,04	0,760599484	0,039092623	DF_N = (1−S·Σ_{k=1..N−1}(α_k·DF_k))/(1+S·α_N)
SWAP	10Y	3,95	30/360	31/12/2025	31/12/2035	10	1	10	0,0395	0,676772132	0,039042065	DF_N = (1−S·Σ_{k=1..N−1}(α_k·DF_k))/(1+S·α_N)

Year	DF at Year (Excel)
1	0,948557632
2	0,893868579
3	0,885430029
4	0,853177604
5	0,821689689
7	0,760599484
8	0,760599484
9	0,760599484
10	0,676772132
11	0,676772132
12	0,676772132
13	0,676772132
14	0,676772132
15	0,676772132
16	0,676772132
17	0,676772132
18	0,676772132

VBA Results - Inputs Preprocessing (values written by macro)

Run the macro CalcInputs_VBA_Outputs to compute derived columns with VBA and compare against Excel formulas.

Type	Tenor	Quote	DayCount	StartDate	EndDate	Accrual τ (manual)	Coupon α	T (years)	Rate Used (VBA)	DF (VBA)	Zero(cc) (VBA)	ΔDF (Excel−VBA)	Check (VBA vs Excel)
CASH	1M	5,1	ACT/360	46022	46053	0,086111111	0,086111111	0,083333333	0,051000	0,995627536	0,052585	1,11022E-16	OK
CASH	3M	5,2	ACT/360	46022	46112	0,25	0,25	0,25	0,052000	0,987166831	0,051665	1,11022E-16	OK
CASH	6M	5,25	ACT/360	46022	46203	0,502777778	0,502777778	0,5	0,052500	0,974282989	0,052107	-1,11022E-16	OK
FUT	9M	94,8	ACT/360	46203	46295	0,255555556	0,255555556	0,75	0,052000	0,961505647	0,052340	-1,11022E-16	OK
FUT	12M	94,6	ACT/360	46295	46386	0,252777778	0,252777778	1	0,054000	0,948557632	0,052813	0	OK
FUT	15M	94,4	ACT/360	46386	46476	0,25	0,25	1,25	0,056000	0,935461373	0,053372	0	OK
FUT	18M	94,2	ACT/360	46476	46568	0,255555556	0,255555556	1,5	0,058000	0,921796274	0,054286	0	OK
FUT	21M	94	ACT/360	46568	46660	0,255555556	0,255555556	1,75	0,060000	0,907877486	0,055226	0	OK
FUT	24M	93,8	ACT/360	46660	46751	0,252777778	0,252777778	2	0,062000	0,893868579	0,056098	-1,11022E-16	OK
SWAP	3Y	4,2	30/360	46022	47118	3	1	3	0,042000	0,885430029	0,040561	-1,11022E-16	OK
SWAP	4Y	4,1	30/360	46022	47483	4	1	4	0,041000	0,853177604	0,039697	1,11022E-16	OK
SWAP	5Y	4,05	30/360	46022	47848	5	1	5	0,040500	0,821689689	0,039278	0	OK
SWAP	7Y	4	30/360	46022	48579	7	1	7	0,040000	0,760599484	0,039093	0	OK
SWAP	10Y	3,95	30/360	46022	49674	10	1	10	0,039500	0,676772132	0,039042	-1,11022E-16	OK

Figure 2: Curve sheet with the Excel bootstrapping table, a validation box, and the VBA Results table.

5 Conclusion

This project shows a complete and readable way to build a simple curve from cash, futures and swaps. The Excel tables explain the logic with transparent formulas and clear intermediate columns. The VBA macros recompute the same results and write them back into the workbook, which gives a direct validation against Excel. Because the checks are OK and the curve validation shows PASS, the curve construction is consistent and the implementation matches the financial formulas.

A VBA code

A.1 Inputs macro (CalcInputs_VBA_Outputs)

```
Option Explicit

Private Function TenorToMonths(ByVal tenor As String) As Long
    tenor = UCase$(Trim$(tenor))
    Dim N As Long: N = CLng(Val(tenor))
    If Right$(tenor, 1) = "Y" Then
        TenorToMonths = 12 * N
    Else
        TenorToMonths = N
    End If
End Function

Private Function BasisFromDayCount(ByVal dc As String) As Long
    dc = UCase$(Trim$(dc))
    Select Case dc
        Case "ACT/360": BasisFromDayCount = 2
        Case "30/360":  BasisFromDayCount = 0
        Case "ACT/365": BasisFromDayCount = 3
```

```
        Case Else:      BasisFromDayCount = 2
    End Select
End Function

Private Function AccrualManual(ByVal d1 As Date, ByVal d2 As Date,
   ByVal dc As String) As Double
    dc = UCase$(Trim$(dc))
    Select Case dc
        Case "ACT/360"
            AccrualManual = (d2 - d1) / 360#
        Case "30/360"
            AccrualManual = WorksheetFunction.Days360(d1, d2, False) /
               360#
        Case "ACT/365"
            AccrualManual = (d2 - d1) / 365#
        Case Else
            AccrualManual = (d2 - d1) / 360#
    End Select
End Function

Public Sub CalcInputs_VBA_Outputs()
    Dim ws As Worksheet: Set ws = ThisWorkbook.Worksheets("Inputs")

    Dim valDate As Date
    valDate = ws.Range("B3").Value

    Dim lastMain As Long
    lastMain = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row
    If lastMain < 7 Then Exit Sub

    Dim outRow As Long: outRow = 28
    Dim i As Long

    ws.Range("E28:L200").ClearContents

    For i = 7 To lastMain
        Dim typ As String, tenor As String, dc As String
        typ = UCase$(Trim$(ws.Cells(i, "A").Value))
        If Len(typ) = 0 Then Exit For

        tenor = Trim$(ws.Cells(i, "B").Value)
        dc = Trim$(ws.Cells(i, "D").Value)

        Dim basis As Long: basis = BasisFromDayCount(dc)
        Dim tenM As Long: tenM = TenorToMonths(tenor)

        Dim startD As Date
        If typ = "FUT" Then
            startD = ws.Cells(i - 1, "I").Value
        Else
            startD = valDate
        End If

        Dim endD As Date
```

```
        If typ = "FUT" Then
            endD = DateAdd("m", 3, startD)
        Else
            endD = DateAdd("m", tenM, valDate)
        End If

        Dim tauYearFrac As Double
        tauYearFrac = WorksheetFunction.YearFrac(startD, endD, basis)

        Dim tauMan As Double
        tauMan = AccrualManual(startD, endD, dc)

        Dim alpha As Double
        If typ = "SWAP" Then alpha = 1# Else alpha = tauMan

        Dim Tyears As Double
        Tyears = tenM / 12#

        ws.Cells(outRow, "E").Value = basis
        ws.Cells(outRow, "F").Value = tenM
        ws.Cells(outRow, "G").Value = startD
        ws.Cells(outRow, "H").Value = endD
        ws.Cells(outRow, "I").Value = tauYearFrac
        ws.Cells(outRow, "J").Value = tauMan
        ws.Cells(outRow, "K").Value = alpha
        ws.Cells(outRow, "L").Value = Tyears

        outRow = outRow + 1
    Next i

    ws.Range("G28:H200").NumberFormat = "dd/mm/yyyy"
    ws.Range("I28:L200").NumberFormat = "0.000000000"
End Sub
```

Explanation of the Inputs VBA macro (natural description)

In the Inputs sheet, I wanted to prove that I can rebuild all the derived columns without using Excel formulas. So I wrote the macro `CalcInputs_VBA_Outputs` and I made it write the results as values in the "VBA Results" table. My idea was simple: the Excel table shows the logic with formulas, and the VBA table shows that I can reproduce exactly the same logic programmatically, line by line.

First, I read the valuation date from cell `B3`. Then I loop over each instrument row in the main table (starting at row 7) until the first empty line. I also clear the previous outputs in the VBA table area so the results are always clean.

I created a small function `TenorToMonths` because I wanted a clear and robust way to convert tenors into months. If the tenor ends with "Y" I multiply by 12, otherwise I keep the number of months. This is important because I reuse this value both to build the dates and to compute the maturity in years.

I also created `BasisFromDayCount` because Excel `YEARFRAC` needs a numeric basis code. So I map ACT/360 to 2, 30/360 to 0, and ACT/365 to 3. With that, I compute an accrual reference

using `YearFrac(startD, endD, basis)`. I keep this value to show that the VBA results are consistent with the Excel reference.

At the same time, I compute a second accrual called `tauMan`. I did this with the function `AccrualManual` because I wanted to show the real day-count formulas. For ACT/360 and ACT/365 I use the day difference divided by 360 or 365. For 30/360 I use `Days360` and divide by 360. This manual accrual is also the one I use later in the curve bootstrapping formulas.

For dates, I treated futures in a specific way. If the instrument is a future, I take the start date as the previous instrument end date, and I set the end date to start date plus 3 months. If the instrument is not a future, I set the end date to valuation date plus the tenor in months. This creates a consistent quarterly sequence for futures.

Finally, I compute the coupon factor α. For swaps I set $\alpha = 1$ because the fixed leg is annual in my project. For cash and futures, I set $\alpha = \tau$ because the payoff is proportional to the accrual. I also compute T in years as `TenorMonths/12`. At the end of each loop, I write all computed values into the VBA Results table and apply simple number formats so the output is clean and directly comparable to Excel.

A.2 Curve macro (CalcCurve_VBA_Outputs)

```
Option Explicit

Private Function RateUsedFromQuote(ByVal typ As String, ByVal quote As
   Double) As Double
    typ = UCase$(Trim$(typ))
    If typ = "FUT" Then
        RateUsedFromQuote = (100# - quote) / 100#
    Else
        RateUsedFromQuote = quote / 100#
    End If
End Function

Private Function DFAtYear(ByVal y As Long, ByRef Tlist() As Double,
   ByRef DFlist() As Double, ByVal nPts As Long) As Double
    Dim i As Long
    DFAtYear = 0#
    For i = 1 To nPts
        If Tlist(i) <= CDbl(y) Then DFAtYear = DFlist(i)
    Next i
End Function

Public Sub CalcCurve_VBA_Outputs()
    Dim ws As Worksheet: Set ws = ThisWorkbook.Worksheets("Curve")

    Dim lastMain As Long
    lastMain = ws.Cells(ws.Rows.Count, "A").End(xlUp).Row
    If lastMain < 7 Then Exit Sub

    Dim outRow As Long: outRow = 28
    ws.Range("J28:L200").ClearContents

    Dim Tlist() As Double, DFlist() As Double, nPts As Long
    ReDim Tlist(1 To 500)
```

```
    ReDim DFlist(1 To 500)
    nPts = 0

    Dim i As Long
    Dim prevDF As Double: prevDF = 1#

    For i = 7 To lastMain
        Dim typ As String
        typ = UCase$(Trim$(ws.Cells(i, "A").Value))
        If Len(typ) = 0 Then Exit For

        Dim quote As Double: quote = CDbl(ws.Cells(i, "C").Value)
        Dim tau As Double:   tau = CDbl(ws.Cells(i, "G").Value)
        Dim alpha As Double: alpha = CDbl(ws.Cells(i, "H").Value)
        Dim T As Double:     T = CDbl(ws.Cells(i, "I").Value)

        Dim rUsed As Double
        rUsed = RateUsedFromQuote(typ, quote)

        Dim df As Double
        If typ = "CASH" Then
            df = 1# / (1# + rUsed * tau)
            prevDF = df
        ElseIf typ = "FUT" Then
            df = prevDF / (1# + rUsed * tau)
            prevDF = df
        ElseIf typ = "SWAP" Then
            Dim N As Long: N = CLng(Fix(T))

            Dim sumPrev As Double: sumPrev = 0#
            Dim k As Long, dfk As Double
            For k = 1 To N - 1
                dfk = DFAtYear(k, Tlist, DFlist, nPts)
                sumPrev = sumPrev + (1# * dfk)
            Next k

            df = (1# - rUsed * sumPrev) / (1# + rUsed * alpha)
            prevDF = df
        Else
            df = 0#
        End If

        Dim z As Double
        If df > 0# And T > 0# Then
            z = -Log(df) / T
        Else
            z = 0#
        End If

        ws.Cells(outRow, "J").Value = rUsed
        ws.Cells(outRow, "K").Value = df
        ws.Cells(outRow, "L").Value = z

        nPts = nPts + 1
```

```
        Tlist(nPts) = T
        DFlist(nPts) = df

        outRow = outRow + 1
    Next i

    ws.Range("J28:J200").NumberFormat = "0.000000"
    ws.Range("K28:K200").NumberFormat = "0.000000000"
    ws.Range("L28:L200").NumberFormat = "0.000000"
End Sub
```

Explanation of the Curve VBA macro (natural description)

In the Curve sheet, my goal was to bootstrap the curve in VBA with the same steps as in Excel, and then compare the results directly in the "VBA Results" table. I wrote the macro `CalcCurve_VBA_Outputs` so that it computes the key outputs used in the curve construction: `RateUsed`, the discount factor DF, and the continuously compounded zero rate $z(T)$. Excel stays the main reference with formulas, and VBA becomes an independent calculation that validates the same financial logic.

To start, I clear the previous VBA outputs in the output range, and I initialise `prevDF` to 1, because the discount factor at time 0 is 1. Then I loop over each curve instrument in the main table (starting at row 7) until I reach an empty type.

The first key step is converting the market quote into a rate consistent with the bootstrapping formulas. I did this with the function `RateUsedFromQuote`. If the instrument is a future, the quote is a price P and I convert it into a forward rate with $(100 - P)/100$. Otherwise, for cash and swaps, I treat the quote as a percentage rate and I divide by 100. This gives me one variable `rUsed` that I can use everywhere.

Then I compute the discount factor depending on the type. For cash, I apply the money market formula $DF = 1/(1 + r\tau)$ using the manual accrual τ from the sheet. For futures, I chain the curve with $DF_i = DF_{i-1}/(1 + f_i\tau_i)$, where `prevDF` is exactly DF_{i-1}. This is why I update `prevDF` after each cash or future line.

For swaps, I use the standard par swap equation. The main difficulty is the sum of previous discounted fixed coupons. Because the fixed leg is annual, I need discount factors at integer years (1, 2, 3, and so on). I store each computed point in arrays `Tlist` and `DFlist`, and I use the helper function `DFAtYear`. This function returns the last discount factor available with $T \leq y$, which is the same spirit as an Excel LOOKUP. With this, I approximate the coupon sum year by year and compute the last discount factor with

$$DF_N = \frac{1 - S\sum_{k=1}^{N-1}(\alpha_k DF_k)}{1 + S\alpha_N}.$$

Because coupons are annual in my project, I use $\alpha_k = 1$ and α_N comes from the sheet (equal to 1 for swaps). This keeps the VBA logic consistent with the spreadsheet design.

Finally, once I have DF and T, I compute the continuously compounded zero rate with $z(T) = -\ln(DF)/T$. I write `RateUsed`, DF, and z into the VBA Results table at the matching row, and I store the point so that the next swap maturities can reuse the already computed discount factors. The final result is a full VBA curve that matches the Excel curve, which is confirmed by the "OK" checks and the PASS validation summary.